

Week 14 : Mobile Frontend 、 Responsive Web / PWA與Permissions

日期：2026-12-09

本章把Week 11、12完成的即時事件、歷史資料、命令與統計整理成手機可用的前台。學生會在真實手機上驗證版面與操作狀態，區分viewer與operator權限，完成一筆可追蹤的實體命令，並分別判定Responsive Web與PWA條件是否真的成立。

在手機查看裝置、歷史資料與命令結果

先啟動上週已成功的系統，在手機看到自己的裝置，再改一行網頁標題並確認手機也更新。標題來自筆電的index.html；裝置資料則由 API 與 WebSocket 更新，兩者的更新方式不同。

順序	動作	應看到的結果
1	第三節：重新啟動同一套系統	手機 connected，Device ID 對應自己的資料
2	第三節最後：修改一行標題	手機重新整理後顯示新標題
3	第四至七節：理解並測試畫面	能區分新資料、歷史資料、等待、失敗與離線
4	第八節：測試權限	無 key 不能送命令，正確 key 才能建立命令
5	第九節：檢查 PWA 條件	分清手機網頁與實際可安裝 PWA，不以檔案存在代替測試

一、Unit Overview

教學目標

完成本單元後，學生應能：

1. 在手機上完成核心監看與控制流程 (monitor-and-control workflow)，不依賴橫向整頁捲動、僅限桌面的操作方式，並避免誤觸啟動。
2. 在介面上明確區分載入中 (loading)、無資料 (empty)、即時更新 (live)、等待結果 (pending)、完成 (done)、遭拒 (rejected)、逾時 (timeout)、失敗 (failure)、連線中斷 (disconnected) 及離線 (offline) 狀態。
3. 結合HTTP歷史查詢介面 (historical HTTP API) 與網頁雙向通訊更新 (WebSocket update)，不將過期資料 (stale data) 呈現為裝置目前狀態。
4. 在介面與後端 (backend) 同時落實檢視者 (viewer) 及操作者 (operator) 的能力限制，並解釋隱藏或停用按鈕為何不能取代授權邊界 (authorization boundary)。
5. 檢查網頁應用程式資訊清單 (web app manifest) 與服務工作者 (service worker)，依瀏覽器實際條件區分響應式網頁 (responsive web page) 與可安裝的漸進式網頁應用程式 (PWA)。
6. 執行行動裝置易用性 (mobile usability)、權限 (permission)、網路中斷 (network loss) 及實體結果 (physical result) 測試，並保留可重現的證據。

教學內容

本單元發展全端物聯網系統 (full-stack IoT system) 面向使用者的介面。學生會運用行動優先 (mobile-first) 的響應式版面 (responsive layout)、明確的介面狀態 (interface state)、歷史查詢介面 (historical API) 載入與網頁雙向通訊更新 (WebSocket update)，呈現裝置資訊，也呈現資料的不確定性。遠端命令 (remote command) 必須作為可追蹤的操作處理，不能將按下按鈕直接當成實體已動作；操作者授權 (operator authorization) 由後端檢查，並在介面上反映其限制。學生會檢查網頁應用程式資訊清單 (web app manifest)、服務工作者 (service worker)、安全環境 (secure context) 要求與安裝條件，分開回報響應式網頁行為及已驗證的漸進式網頁應用程式 (PWA) 行為。

本週器材外觀

沿用 Week 12 的輸入與 RGB 輸出，從手機核對監看、權限與命令結果；不新增硬體。另備筆電、USB 資料線、區域網路與手機。

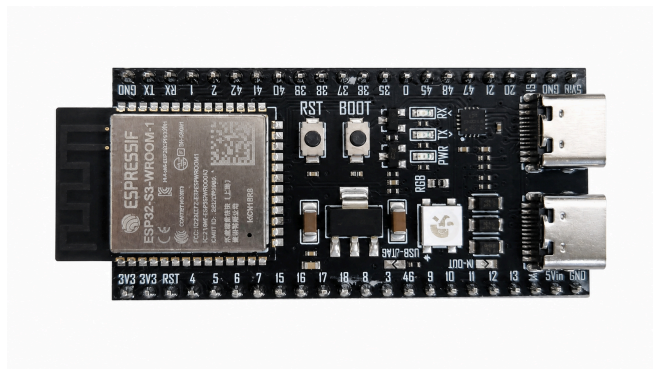
照片下方標示拍攝角度與來源。先辨認零件，再依本週器材表與接線步驟操作；照片本身不是接線指令，也不表示已完成電氣驗證。

[ESP32-S3 開發板](#) · [400 孔麵包板](#) · [杜邦線](#) · [四腳輕觸按鈕](#) · [KY-018 光敏電阻模組](#) · [HW-479 三色發光二極體模組](#)

ESP32-S3 開發板 (Development Board)

照片中的板卡為 YD-ESP32-S3 Type-A V1.5，搭載 N16R8 模組。正反面白底圖是既有後製展示圖；小字與腳位須核對本人實物及本週接線資料。

實物後製展示圖：正面：模組、按鈕與 USB 接頭



實物後製展示圖：背面：板身與排針

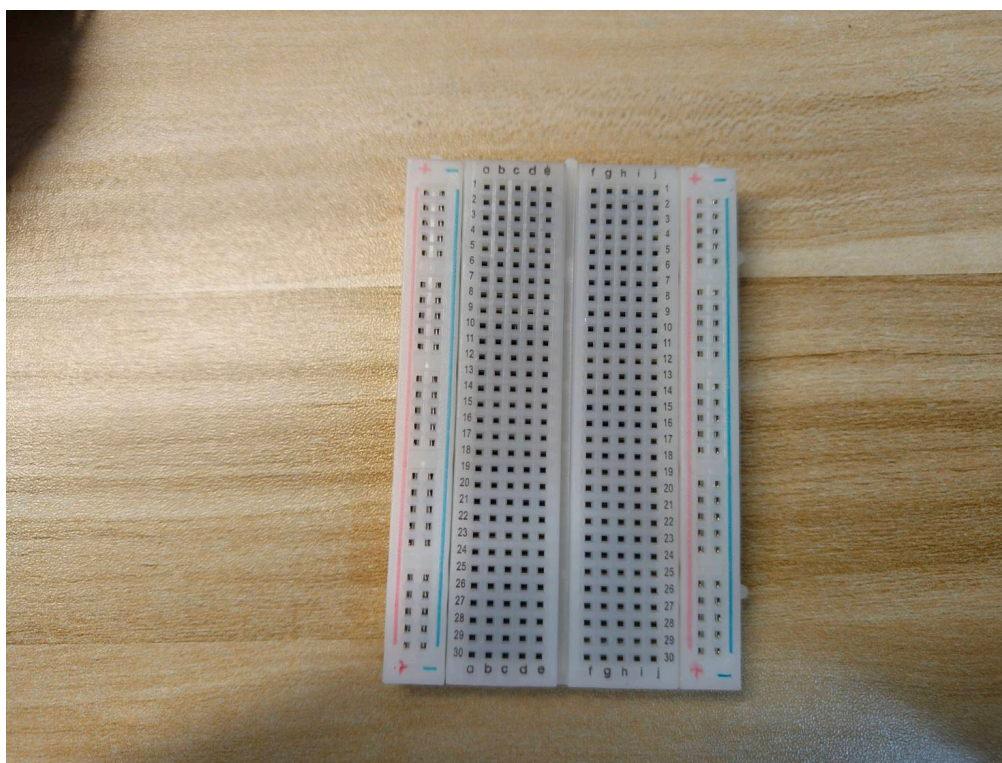


其他留存角度：[麵包板對孔紀錄](#)；不是建議的實驗安裝方式，右側接線空間不足。

400 孔麵包板 (Breadboard)

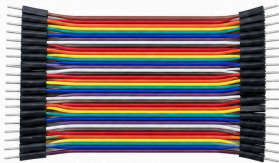
辨認中央溝槽、a~j 字母與列號。外觀照片不表示所有孔都相通，連通關係依本週圖解與斷電量測確認。

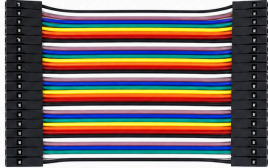
實物照片：俯視：中央溝槽、五孔組與側邊電源軌



杜邦線 (Jumper Wire)

露出金屬針的是公頭 (Male)，有插孔的是母頭 (Female)；綠色不會自行決定電壓或功能。所需接頭種類依當週器材表，不是每週都用完三種。商品參考卡上的數量與金額是歷史資料，不是學生應買數量或目前售價。

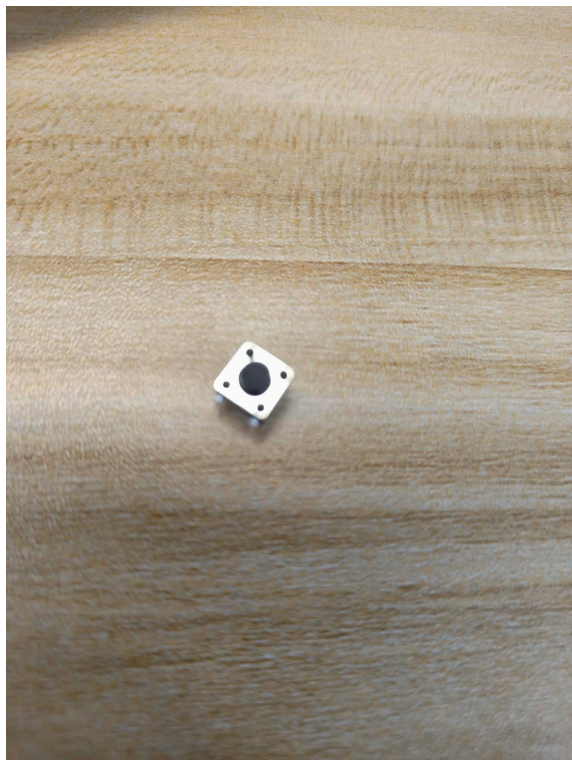
實物照片：成排導線與接頭全貌	蝦皮商品參考：公對公：兩端皆為金屬針
	 【環島科技】40P彩色杜邦線 線長10/20/30CM 一排40根 母頭對母頭 公頭對公頭 公頭對母頭彩色排線 規格：20cm，公對公 購買數量：6排 訂單金額：NT\$150

蝦皮商品參考：公對母：金屬針與插孔各一端	蝦皮商品參考：母對母：兩端皆為插孔
 【環島科技】40P彩色杜邦線 線長10/20/30CM 一排40根 母頭對母頭 公頭對公頭 公頭對母頭彩色排線 規格：20cm，公對母 購買數量：6排 訂單金額：NT\$150	 【環島科技】40P彩色杜邦線 線長10/20/30CM 一排40根 母頭對母頭 公頭對公頭 公頭對母頭彩色排線 規格：20cm，母對母 購買數量：6排 訂單金額：NT\$150

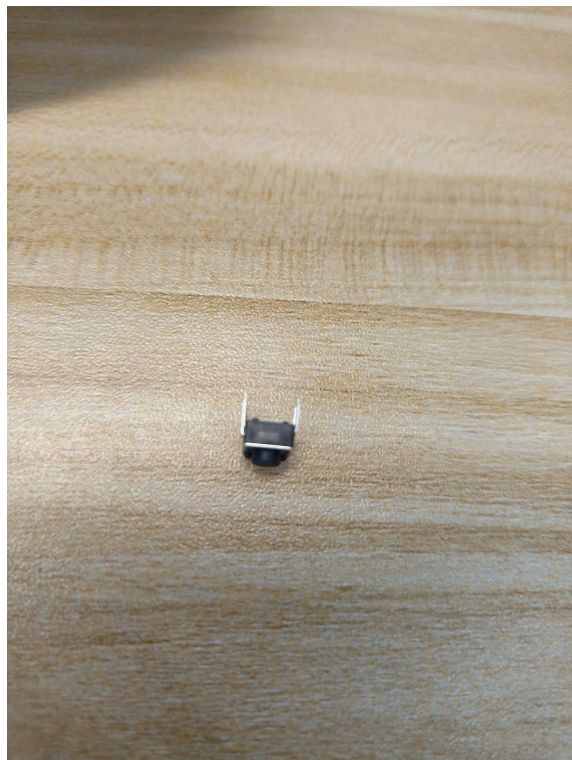
四腳輕觸按鈕 (Tactile Pushbutton)

上方黑色部分是按壓位置，四支金屬腳用來連接電路。照片不能單獨證明哪一對腳常通；先斷電，依 Week 2 的方法辨認。

實物照片：俯視：按鍵與金屬上蓋



實物照片：側面：四支接腳

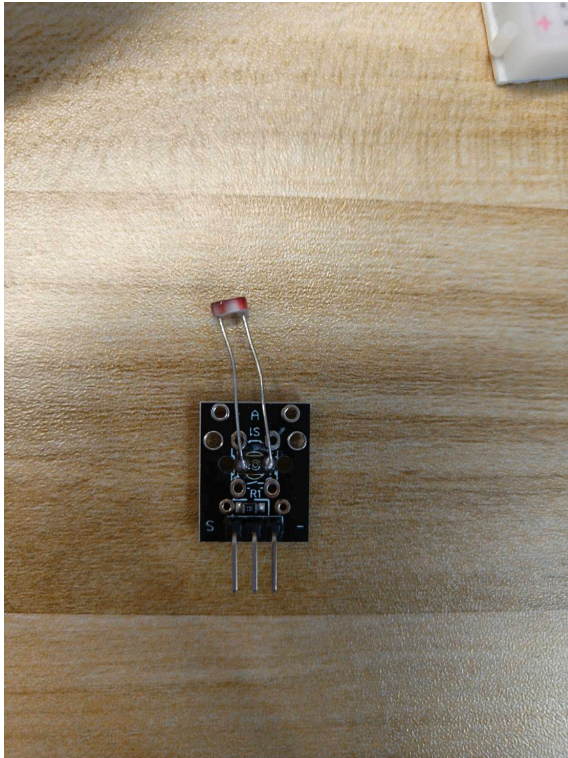


其他留存角度：[歷史接線紀錄](#)：同一組常通接點的量測，不是按下才導通的接法答案。

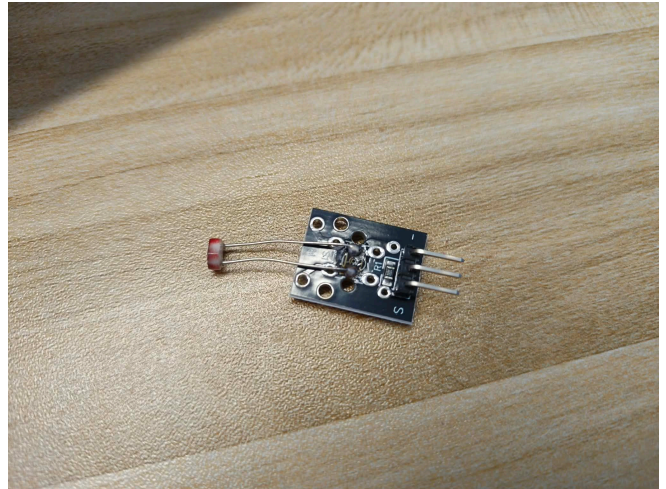
KY-018 光敏電阻模組 (Photoresistor Module)

圓形感光元件、板上固定電阻與三支排針構成模組。S 是訊號標示；元件區的 A、S1、R1 不能直接當成中間排針名稱。接線沿用已確認的 Week 3 紀錄。

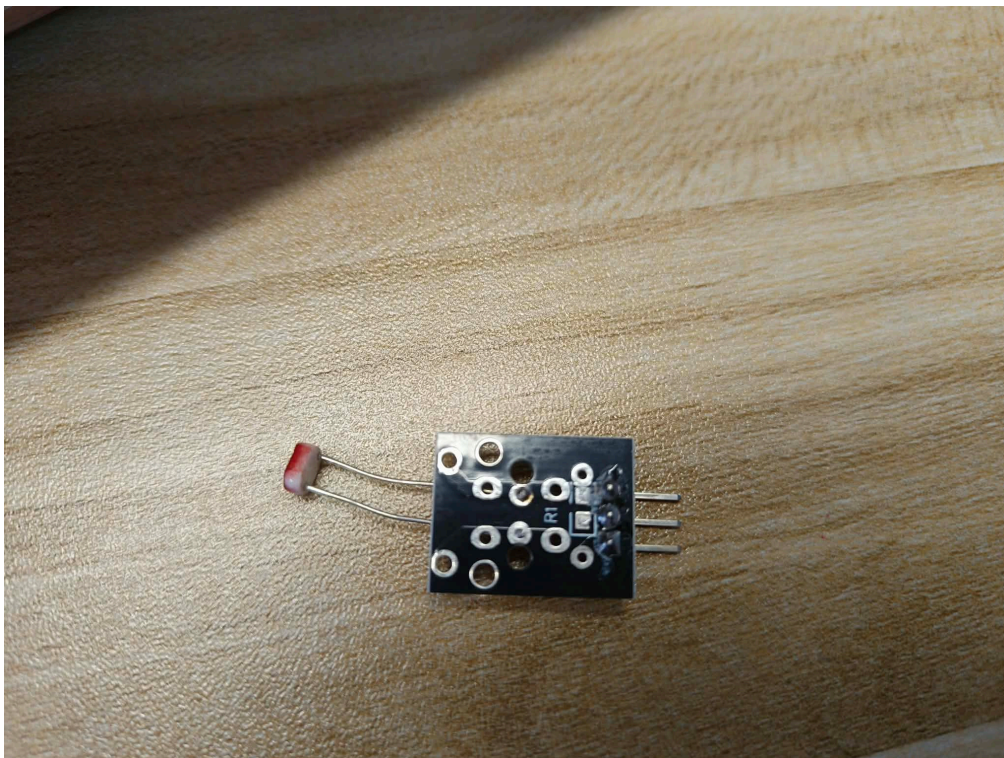
實物照片：正面近照：感光元件、S 與 - 絲印



實物照片：另一元件面角度

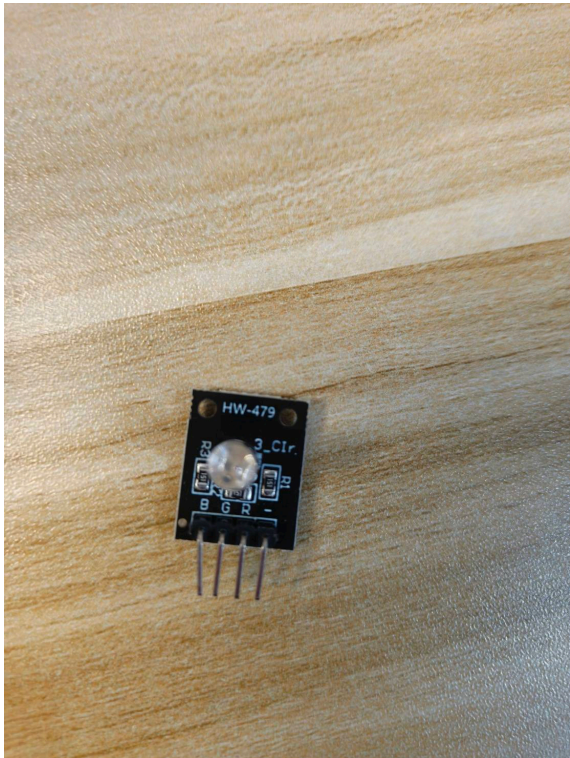
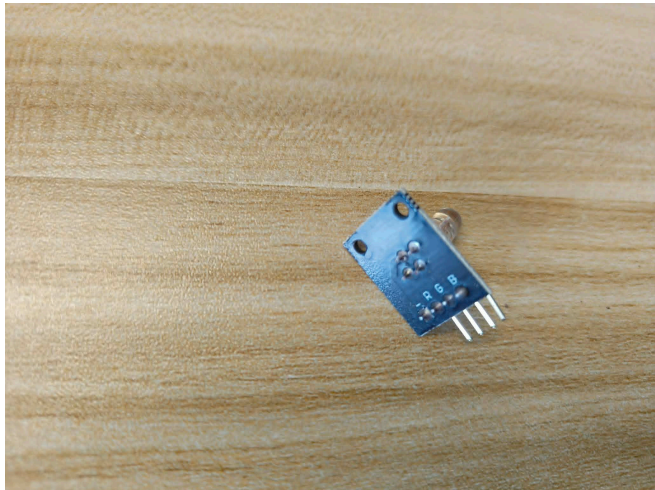


實物照片：焊接面



HW-479 三色發光二極體模組 (RGB LED Module)

訂單稱 KY-016；實物 PCB 標示 HW-479，前方可見 B、G、R、- 與板上電阻。共同端、阻值及控制電流仍須核對，不能用八顆燈條取代這個模組。

實物照片：正面：單顆 LED 與 B / G / R / - 標示	實物照片：焊接面
	

其他留存角度：[較早的元件面照片](#)；[失焦補充照](#)；[不供腳位或焊點判讀](#)。

二、核心使用流程與開始狀態

本週不新增硬體。使用已購ESP32-S3、START / STOP按鈕、KY-018及KY-016 RGB；可沿用 Week 12 MQTT或Week 11 HTTP路徑。只有下列完整路徑先正常，才開始修改前台：

真實輸入 → ESP32 → Backend → Database → 手機顯示
手機命令 → Backend → 目標ESP32 → 實體RGB → result → 手機顯示

最小核心流程：

1. 使用者開啟手機頁面，立即知道網路、WebSocket與角色狀態。
2. 輸入不含個資的device ID，讀到該裝置歷史事件、命令與統計。
3. Viewer可以監看，但不能建立命令。
4. Operator輸入臨時key後送出安全start、stop或reset。
5. 畫面先顯示requested，之後顯示accepted與terminal result；實體RGB結果一致。
6. 裝置、Backend或網路失效時，畫面不把舊資料誤標成「現在正常」。

三、啟動基準系統

先決定本次只使用哪一條路徑：板上是 Week 11 HTTP 程式，或 Week 12 MQTT 程式。不要同時換硬體與修改前台；先重現上次成功結果。

在檔案總管開 IOT_Introduction/examples/course_backend，從此處開 PowerShell 作為後端視窗。先執行 `Test-Path .\app.py` 與 `Test-Path .\.venv\Scripts\python.exe`，兩個都應為 True。若舊後端仍在執行，先在它的視窗按 **Ctrl+C**，不要再啟動第二個占用 8000 的服務。替換臨時 key 後執行：

```
$env:IOT_OPERATOR_KEY="replace-with-your-temporary-classroom-key"
$env:IOT_COMMAND_TIMEOUT_SECONDS="20"
.\.venv\Scripts\python.exe -m uvicorn app:app --host 0.0.0.0 --port 8000
```

若使用 MQTT，照 [Week 12 第四、五節](#) 啟動 A 的 broker 與 E 的 bridge；D 就是這裡已啟動的後端，不要重開。已有密碼檔時跳過建立密碼檔，使用原本設定。新 E 視窗仍須重新填 MQTT 環境變數。若使用 HTTP，不啟動 broker 或 bridge。

ESP32 先保持 IDLE 且 RGB 為安全顏色。筆電開 `http://127.0.0.1:8000`，手機開 `http://<筆電LAN-IP>:8000`，以實際 IP 取代尖括號內容。把頁面 **Device ID** 改成板上 .ino 的裝置代號，按 **套用並重新整理**。先留空 Operator key，看見資料後才做控制；歷史資料不代表裝置現在連線。

Backend命令deadline預設20秒。**timeout (逾時)** 表示裝置在deadline前沒有完成，不是「命令稍後一定會成功」。發生timeout後，不自動重送可能造成危險或重複的動作。

先改一行標題，確認你改的是正在使用的頁面

1. 保持後端執行。在檔案總管開 static 資料夾，把 index.html 複製為 index.before-week14.html 作本機備份；不要改到 app.py。
2. 用文字編輯器開 index.html，不是雙擊用瀏覽器開。按 **Ctrl+F** 搜尋 `<h1>IoT Course Console</h1>`，把中間文字改為 我的 IoT 控制台，保留兩側標籤。
3. 按 **Ctrl+S**。回到手機原本 `http://筆電IP:8000` 網址，重新整理一次，應看到新標題。這次是網頁檔案修改，所以需要重新整理；不要與事件經 WebSocket 自動更新混在一起。
4. 沒變時依序查：是否儲存正確檔案、網址是否仍是正在執行的筆電、後端是否還在執行。不使用 `file://` 開檔取代伺服器網址。後端離線時可能看到快取舊頁，先恢復連線。
5. 要撤回這次文字修改，就把同一個 `<h1>` 的內容改回原文字並儲存，再重新整理手機。

四、閱讀前台的HTML、CSS與JavaScript

本週基準頁面為[static/index.html](#)。沿用剛才的 本機備份，已有 Git 操作習慣者也可另建分支保存自己的修改；不把建立分支當成開始練習的前置障礙。不要修改 app.py 的權限規則來讓畫面看似成功。

4.1 HTML：結構與可辨認控制

HTML描述頁面的內容與語意。開啟index.html，找到：

- `<meta name="viewport" ...>`：告訴手機以裝置viewport寬度排版；缺少時手機可能先以寬桌面畫布縮小整頁。
- `role="status"`：讓連線與命令狀態成為可被輔助技術辨認的狀態訊息。
- 包住input的label：標籤文字與控制項形成可辨認關係，點標籤也能聚焦正確欄位。
- Events與Commands table：寬螢幕用table，窄螢幕由CSS改成card-like rows。
- `type="password"`及`autocomplete="off"`：避免key直接顯示；仍不能把它當成後端安全。

4.2 CSS：Responsive Web

Responsive Web Design（響應式網頁設計）使同一份內容依viewport寬度重新排列。基準CSS先定義一般版面，再以@media (max-width: 680px)調整窄螢幕。@media內隱藏table header，並使用cell的data-label呈現欄名，避免手機只能左右拖曳大型表格。

檢查按鈕min-height: 44px、input寬度、單欄排列、文字換行及錯誤訊息。Responsive不只是「頁面縮小」；核心操作必須可讀、可點、可理解。原理可參考[MDN Responsive Web Design](#)。

4.3 JavaScript：資料與狀態

JavaScript在頁面載入後讀API、更新DOM（Document Object Model，瀏覽器中的頁面物件）及維持WebSocket。找到這些函式：

Function	作用	失敗時必須呈現
fetchJson()	執行HTTP並解析JSON	status與錯誤detail
refresh()	平行讀events、commands、stats	loading、empty或failure
connectWebSocket()	建立即時連線並重連	connecting / connected / disconnected
renderEvents()	以安全text node顯示事件	空array顯示empty
renderCommands()	顯示命令狀態	pending與terminal status
command form的submit handler	帶operator header建立命令	403、offline或其他failure

頁面以textContent建立外部資料，不用innerHTML直接插入裝置payload，降低惡意文字被當HTML執行的風險。

五、先做手機Responsive Baseline Test

1. 手機關閉自動旋轉，先用直向開啟頁面。
2. 不用雙指縮放，從頂端依序找到connection、device ID、operator key、command、stats、events與commands。
3. 確認頁面本身沒有水平捲動；長device ID與reason可換行。
4. 按鈕之間有足夠間距，不會因拇指觸碰相鄰控制而誤送。
5. 切到橫向再測一次；版面可使用較多寬度，但內容順序不改變。
6. 在筆電瀏覽器縮窄視窗，找出table轉成card的breakpoint現象。

完成條件不是一張好看的截圖，而是支援資料中每一個核心任務都能在實際手機完成。

六、驗證「最後成功更新時間」

歷史資料曾成功載入，不代表現在仍連線。基準頁面的裝置摘要已有 `<strong id="last-refresh">`，JavaScript以`ui.lastRefresh`取得它，並且只在 `refresh()`的三個API都成功後執行：

```
ui.lastRefresh.textContent = new Date().toLocaleString();
```

依下列方式確認它真的代表「最後成功」，而不是「最後嘗試」：

1. Backend正常時按「套用並重新整理」，記下顯示日期與時間。
2. 等待數秒後再刷新一次，時間應更新。
3. 停止Backend，再按刷新或等待WebSocket重連；Events與Commands應顯示讀取失敗，保留下方舊資料並明確標示為舊資料；`Last successful refresh`必須保留上一次成功值。
4. 重新啟動Backend，成功refresh後時間才再次更新。

如果在catch或開始loading時更新這個欄位，就會把失敗嘗試誤標成成功；必須移回 `try`中的API與render完成之後。

七、完整介面狀態測試

7.1 Loading與Empty

重新載入頁面，應先看到Loading。使用一個從未出現的device ID，API成功回空array時應顯示「目前沒有符合條件的事件 / 命令」，不能顯示讀取失敗。

7.2 Live與Historical

用真實ESP32 ID讀取歷史資料，再按實體START。不刷新頁面就出現新事件，表示WebSocket live update；重新整理後仍存在，表示historical API / database可讀回。兩者都要通過。

7.3 Pending、Success與Rejected

送start後立即顯示requested；裝置收到顯示accepted；完成後顯示done。進ERROR後再送 start，應顯示rejected與原因，RGB不變綠。

7.4 Timeout

讓ESP32斷線，對正確device ID送安全reset。命令先requested，超過Backend deadline後 變timeout。重新連線時不能自動執行這筆已timeout命令。

7.5 Failure與Offline

- Operator key故意輸錯：HTTP 403，畫面顯示failure，不新增命令。
- 停止Backend：WebSocket disconnected，API讀取失敗，控制按鈕disabled。
- 關閉手機Wi-Fi：browser network offline；舊資料仍可見時必須保留「最後成功更新」與 offline狀態，不能顯示為live。

八、Viewer與Operator Permissions

authentication (身分驗證) 判斷請求者是誰或是否持有credential；**authorization (授權)** 判斷是否允許執行動作。本週簡化原型使用operator key授權建立命令。

Viewer

1. Operator key留空。
2. 可讀events、commands與stats。
3. Send按鈕disabled。

Wrong key

1. 輸入錯誤key；頁面文字可能顯示「operator key entered」，這只代表欄位不空，不代表 Backend已驗證。
2. 送命令得到403。
3. Database沒有新增command。

Valid operator

輸入正確臨時key後，Backend建立command並回201。key只放JavaScript變數，不放URL、localStorage、Git或畫面log。LAN HTTP沒有TLS加密，不能保護傳輸中的key；本週只使用 短期課堂key與低功率輸出，結束後作廢。對外部署必須另做HTTPS、使用者及裝置身分驗證。

即使使用者在Developer Tools移除disabled，Backend仍會拒絕沒有正確header的請求。因此UI restriction改善使用流程，Backend authorization才是安全邊界。

九、Manifest、Service Worker與PWA判定

Web App Manifest是manifest.json，描述app name、start URL、display與icon。Service Worker是與頁面分開執行的script，可攔截network request並提供cache等能力。

基準檔案：

- [manifest.json](#)
- [sw.js](#)
- [icon.svg](#)

9.1 筆電localhost觀察

1. 用筆電開http://127.0.0.1:8000。
2. 在筆電 Chrome / Edge 按 F12 (或 Ctrl+Shift+I) 開開發者工具。頂部選 **Application**；看不到就按 » 展開更多分頁。左側依序選 **Manifest**、**Service Workers**。這不是 Arduino IDE，也不是手機設定頁。
3. 重新整理，確認shell檔可由service worker cache取得；/api/資料不可由舊cache冒充最新值。
4. 看到 service worker 已啟用後，停止Backend，再重新整理。若只有頁面框架可開，也必須顯示 API 失敗 / offline；框架不是最新裝置資料。完成後在原後端視窗重跑第三節啟動命令，再重新整理確認恢復。

9.2 手機LAN HTTP限制

一般http://<筆電LAN-IP>:8000不是HTTPS，也不是手機自己的localhost。Service Worker 只在secure context可用，PWA installability也通常要求HTTPS或localhost / loopback。因此本週共同LAN流程可以驗證 **Responsive Web**，但不能因repository有manifest與sw檔就 宣稱手機PWA已安裝。官方條件見[MDN Service Worker API](#) 及[MDN Making PWAs installable](#)。

判定用語：

- Responsive Web verified on phone over LAN HTTP：手機核心流程通過。
- Manifest and service worker inspected on desktop localhost：筆電local開發條件通過。
- Installable PWA verified on phone over HTTPS：只有真的配置HTTPS並完成安裝 / 離線測試才可寫。

十、真實命令端到端驗收

1. ESP32為IDLE，手機顯示WebSocket connected及最近成功更新時間。
2. Operator對正確device送start。
3. 記錄command_id與requested time。
4. ESP32回accepted，RGB變綠，再回done。
5. 手機顯示同一ID的done；Events另有remote_start或相應事件。
6. 送stop，RGB進ERROR安全輸出並回done。
7. 停止Backend，按實體STOP仍生效；前台只顯示offline / stale，不能顯示新done。

核心驗收至少在一支真實手機完成；只用desktop responsive preview不能取代phone test。

十一、練習

練習1：狀態文字而非只有顏色

檢查connected、offline、pending、done、rejected、timeout是否都有文字。選一個只靠顏色的 地方改成「文字 + 顏色」，並在灰階或降低螢幕亮度後重新辨認。

練習2：危險操作確認

為start加入清楚確認，內容要包含target device及動作；stop不可被確認dialog延遲。完成後測試取消不建立命令、確認才建立命令、STOP仍可立即送出。

操作	預期結果
選 start，按 Send	確認視窗顯示裝置代號及 start，尚未建立 command
按取消	不新增 command，RGB 不因這次取消而改變
再送 start，按確定	建立一筆 command；依同一 ID 追蹤裝置結果
選 stop，按 Send	不出現確認視窗，直接送請求；仍需等待裝置回報

「直接送」不表示零網路延遲，也不能取代本機 STOP 或斷電。

練習2參考做法

1. 備份並用文字編輯器開啟 `static/index.html`。
2. Ctrl+F 搜尋 `#command-form`，找到 `submit` 事件處理函式。
3. 在檢查 `deviceId`、`operatorKey`、`network`、`wsConnected` 的 `return` 那一行之後，且在 `ui.sendButton.disabled = true;` 之前，加入：

```
if (command === 'start') {  
  const confirmed = window.confirm(  
    '裝置：' + deviceId + '\n動作：start\n確定送出？'  
  );  
  if (!confirmed) return;  
}
```

4. 儲存，重新整理手機頁面，再填本次臨時 key。依表測取消、確認及 `stop`，對照 `Commands` 中新增的 ID；不要只看彈出視窗。

取消時在送出 HTTP 前 `return`，因此不建立命令；`stop` 不符合這個 `if` 條件，不會被確認視窗攔住。原有後端權限、錯誤顯示及裝置拒絕條件均保留，按確定不代表硬體已成功。

練習3：一個歷史filter

加入 `event type` 或時間範圍其中一種 `filter`。空結果顯示 `empty`，無效輸入顯示 `validation message`，`network failure` 顯示 `failure`；三者不可共用相同訊息。

十二、繳交內容與完成條件

繳交手機直向與橫向核心流程、`viewport / responsive breakpoint`觀察、最後成功更新功能、完整狀態矩陣、`viewer / wrong key / valid operator`測試、`command ID`實體結果、`offline / timeout` 測試、`localhost service worker`觀察及PWA判定用語。截圖不得含 `operator key` 或個資。

- ☐ 手機不縮放、不水平拖動即可監看與操作。
- ☐ `Loading`、`empty`、`failure`、`offline`與`stale`可分辨。
- ☐ `Requested`、`accepted`、`done`、`rejected`與`timeout`可分辨並帶`command ID`。
- ☐ 歷史API與Websocket live update都使用真實裝置資料。
- ☐ `Viewer`不能送命令，`wrong key`被Backend 403拒絕，`valid operator`才可建立命令。
- ☐ `Key`不在URL、`localStorage`、`log`、`Git`或截圖。
- ☐ 離線時控制`disabled`，舊資料保留最後成功時間而非冒充`live`。
- ☐ 手機命令與ESP32實體RGB結果一致，本機STOP不依賴Backend。
- ☐ `Responsive Web`、`localhost service worker`與`installable phone PWA`分開標示。
- ☐ 未配置與測試HTTPS時，不宣稱手機PWA已安裝或`offline data`已完整可用。

完成後使裝置回IDLE，停止Backend / bridge / broker並拔USB。詳細手機測試表、權限矩陣、狀態矩陣、PWA證據表及延伸實作見[Week 14支援資料](#)。

準備、紀錄表與延伸參考

一、手機與瀏覽器測試環境

項目	實際值	取得方式	注意事項
手機作業系統 / 版本		Settings	不記裝置序號或帳號
瀏覽器 / 版本		Browser About	安裝支援因browser而異
Viewport直向		DevTools或實測	不用型號名稱代替寬度
Viewport橫向		DevTools或實測	旋轉後重測核心流程
Backend URL		LAN IP與port	截圖可局部遮IP
Transport	HTTP / HTTPS	address bar	決定secure context條件
WebSocket	ws / wss	page / DevTools	應與頁面protocol一致
Device ID		程式profile	不含個資

二、手機核心流程表

步驟	使用者動作	預期畫面回饋	Backend證據	ESP32 / 實體證據	實際結果
1	開啟URL	network、 WebSocket、role 可見	GET與WS連線	無非預期動作	
2	輸入device ID	loading後events / stats	filtered GET	ID一致	
3	Viewer嘗試操作	button disabled	無command row	輸出不變	
4	輸入operator key	role提示key entered	尚未驗證	輸出不變	
5	送start	requested / accepted / done	同command ID	RGB變綠	
6	送stop	requested / accepted / done	同command ID	RGB安全紅色	
7	Backend停止	disconnected / failure / stale	程序停止	本機STOP仍有效	

三、Interface State Matrix

State	如何建立	必須顯示	按鈕狀態	不可顯示	實際證據
Loading	第一次refresh	Loading文字	暫停送出	Empty / success	
Empty	未知device、API成功	沒有符合資料	依權限	Failure	
Live	WS connected	Connected + 更新時間	依權限	Offline	
Pending	command created	command ID / requested	防重複送出	Done	
Accepted	device收到	accepted	視動作決定	Physical done	
Done	device完成	done與message	可下一步	Pending	
Rejected	安全規則拒絕	rejected與reason	保持安全	Done	
Timeout	超過deadline	timeout與ID	不自動重送	Success	
Failure	API 403 / 500	status與detail	視原因停用	Empty	
Disconnected	WS關閉	disconnected/retrying	停用remote	Connected	
Offline	browser offline	offline + last success	停用remote	Live	
Stale	舊資料仍在	last success time	停用或警告	Current/now	

四、Responsive檢查表

直向手機

- ☐ Address bar下不需雙指縮放即可讀第一層狀態。
- ☐ 頁面沒有整體水平scroll。
- ☐ Device ID、key與command控制完整可見。
- ☐ Input有label，聚焦後software keyboard不遮住必要動作。
- ☐ Button高度與間距足以避免相鄰誤觸。
- ☐ Long ID、reason、timestamp能換行或安全截斷。
- ☐ Events / commands每個value仍有欄名，不只剩數字。
- ☐ Error不只靠紅色，success不只靠綠色。

橫向手機 / 寬螢幕

- ☐ 旋轉後不遺失device、key或pending state。
- ☐ Layout使用增加寬度，但閱讀順序一致。
- ☐ Table header與cell對齊。
- ☐ Zoom至200%仍能使用核心操作。
- ☐ Keyboard-only desktop可依合理tab order操作。

五、Permission Test Matrix

Case	UI state	Request header	預期HTTP	Database新增	ESP32動作	實際
Viewer / 空key	disabled	無	不應送出	否	否	
DevTools移除disabled / 空key	可點	無	403	否	否	
Wrong key	可點	錯誤key	403	否	否	
Valid key / 錯device	可點	正確key	201	是	無，最後timeout	
Valid key / 正確device	可點	正確key	201	是	依命令	
Offline / valid key	disabled	不送出	無	否	否	

UI顯示operator key entered只是local input state。只有Backend的HTTP result能證明授權成功。

六、Command Trace

時間	層次	command_id	status	message	physical state
	Mobile request		requested		IDLE
	Backend		requested		IDLE
	Device		accepted		IDLE / processing
	Device result		done / rejected		
	WebSocket UI		terminal		

若手機顯示done但實體不符，介面驗收失敗；不能以Backend row取代physical evidence。

七、Responsive Web與PWA證據表

Capability	測試URL / context	需要條件	觀察方法	結果	可使用的結論
Responsive layout	Phone LAN HTTP	viewport、CSS	真實手機 workflow		
Manifest可讀	Desktop localhost	manifest link	DevTools Application		
Service worker registered	Desktop localhost	localhost secure context	DevTools		
Shell cache	Desktop localhost	active SW	offline reload		
API freshness	Online / offline	/api/不以舊 cache冒充	關Backend		
Phone installability	Phone	HTTPS或符合 browser條件	install UI / installed launch		
Phone offline behavior	Installed PWA	active SW與設計策略	airplane/offline test		

Repository中存在manifest、icon與sw，只能證明檔案存在。安裝按鈕、installed icon、standalone launch及offline behavior必須實際測試後才能標示通過。

八、Fault Injection表

故障	唯一變因	UI預測	Backend / WS 觀察	實體預測	實際	復原證據
Wrong key		403 failure	denied log	不動作		
Unknown device		pending→timeout	command row	不動作		
ESP32 offline		timeout	no ack	不動作		
Backend stopped		disconnected / failure	process off	STOP仍本機有效		
Phone network off		offline / stale	WS close	裝置保持安全		
Malformed API data		readable failure	422	不動作		

九、Usability Observation表

請另一位同學只依頁面文字完成核心流程。不得提供口頭操作步驟；可以阻止不安全動作。

Task	第一次找到控制所需步驟	誤觸 / 疑問	完成 / 失敗	要修改的label、order或feedback
找到connection state				
選擇device				
辨認viewer限制				
送安全command				
找terminal result				
判斷offline / stale				

十、故障排查表

症狀	第一個檢查	正常基準	下一步
手機頁面完全打不開	筆電localhost是否可開	Backend running	查LAN IP / firewall
手機版像縮小桌面	檢查viewport meta	device-width	再查CSS fixed width
頁面水平scroll	找出超出viewport元素	main width flexible	查table、long text、fixed px
Loading不結束	Network panel第一個failed request	API 200	顯示catch failure
Empty與failure相同	比較HTTP 200空array與exception	不同文字	分開render路徑
WS connected但歷史空	直接測GET API	history independently works	查filter ID
History有資料但live不更新	看WS status	connected	查message handler
Button一直disabled	讀network、WS、key三條件	全部ready	不繞過Backend權限
Wrong key看似operator	實際送安全命令看403	Backend決定	改UI文字，不宣稱validated
Pending不結束	查device及timeout worker	terminal status	不自動重送
Offline仍能點控制	查updateAvailability()	disabled	先停用再修狀態
Service worker在手機LAN HTTP失敗	看protocol與context	HTTPS / localhost	記為responsive only

十一、Lab Notebook模板

```
日期 / 組別 / commit :
Phone / browser :
URL protocol :

Responsive :
- portrait :
- landscape :
- horizontal scroll :
- tap / keyboard / text :

States :
- loading / empty / live :
- requested / accepted / done :
- rejected / timeout / failure :
- disconnected / offline / stale :

Permissions :
- viewer :
- wrong key :
- valid operator :
- key exposure check :

Physical command trace :
- command_id :
- phone result :
- ESP32 / RGB result :

PWA evidence :
- manifest :
- service worker context :
- phone installability :
- exact conclusion :
```

十二、延伸實作

延伸A：Client-side event filter

加入event type select，選項由實際資料建立。Filter後0筆顯示empty；API failure仍顯示failure。

延伸B：Command confirmation與cooldown

START確認顯示target device；送出後短暫停用重複START，直到terminal result或timeout。STOP 不套用會延遲安全停止的confirmation / cooldown。

延伸C：Accessible live region

以aria-live讓重要command result可被screen reader讀出，但避免每兩秒telemetry造成大量打斷。記錄使用哪類訊息觸發announcement。

延伸D：HTTPS PWA deployment

只在已核准的hosting環境配置HTTPS、authentication與secret management，再做phone install、standalone launch、service worker update與offline測試。不可直接公開課堂Backend port。

十三、官方與Repository參考

- [MDN Responsive Web Design](#)
- [MDN Service Worker API](#)
- [MDN Making PWAs installable](#)
- [課程mobile page](#)
- [課程manifest](#)
- [課程service worker](#)
- [Week 12 Database主教材](#)